

Arquitetura e Organização de Processadores

Projeto: FPU IEEE 754 em Verilog

1. Objetivo e escopo

Projetar, descrever em Verilog e verificar uma unidade de ponto flutuante de precisão simples (IEEE 754 binary32) capaz de executar as operações de soma, subtração e multiplicação. Ao final do projeto, a equipe deverá entregar um módulo sintetizável `fpu_top`, que recebe dois números de 32 bits e um código de operação, produzindo o resultado no mesmo formato, juntamente com as flags de overflow e underflow.

Os operandos de entrada serão sempre números normalizados, com expoente entre 1 e 254. Não se deve implementar o tratamento de NaN, infinito, zero ou números subnormais como operandos de entrada.

2. Formato binary32

Os números utilizam o formato IEEE 754 de precisão simples:

- 1 bit de sinal;
- 8 bits de expoente, com bias 127;
- 23 bits de fração.

A mantissa possui 24 bits internamente, considerando o bit implícito 1.

O campo de expoente da entrada estará sempre entre 1 e 254.

3. Operações

op	Operação
00	Soma
01	Subtração
10	Multiplicação
11	Reservado

Soma e subtração

A implementação deverá realizar o alinhamento dos expoentes e a soma ou subtração das mantissas, considerando os bits guard, round e sticky.

O resultado deverá ser normalizado antes do empacotamento no formato binary32.

Multiplicação

A multiplicação deverá utilizar as mantissas de 24 bits, produzindo um resultado interno de 48 bits. O resultado deverá ser normalizado e arredondado antes do empacotamento.

É permitido utilizar o operador `*` do Verilog para realizar a multiplicação.

4. Arredondamento e limites do expoente

O projeto utilizará o arredondamento simplificado round half up.

Após a normalização, deverão ser considerados os bits:

- G – guard;
- R – round;
- S – sticky.

O bit sticky deve representar o OR dos bits descartados.

Os limites do expoente deverão ser tratados da seguinte forma:

- overflow: resultado infinito ($E = 255$, $F = 0$) e overflow = 1;
- underflow: resultado zero e underflow = 1;
- zero exato: a subtração de dois valores iguais deve produzir zero.

Não se deve implementar modos configuráveis de arredondamento.

5. Interface do módulo

A interface de fpu_top deve ser mantida conforme especificado:

```
module fpu_top (
    input  wire      clk,
    input  wire      rst_n,
    input  wire      start,
    input  wire [1:0] op,
    input  wire [31:0] a,
    input  wire [31:0] b,
    output reg [31:0] result,
    output reg      done,
    output reg      overflow,
    output reg      underflow
);
```

Requisitos do protocolo:

- start inicia uma operação;
- a, b e op devem estar estáveis quando a operação for iniciada;
- done deve permanecer em 1 durante um ciclo quando o resultado estiver disponível;
- result, overflow e underflow devem ser válidos junto com done;
- um novo start durante uma operação em andamento deve ser ignorado;
- as flags devem ser zeradas no início de uma nova operação.

A implementação de referência utiliza dois ciclos de latência, mas outra latência pode ser utilizada desde que respeite o protocolo start/done.

6. Arquitetura

A implementação deverá apresentar uma estrutura modular, com separação adequada entre as principais funções do datapath e do controle.

A divisão dos módulos fica a critério da equipe. A solução deverá:

- evitar a implementação de toda a FPU em um único bloco;
- separar adequadamente lógica combinacional e sequencial;
- permitir identificar os principais caminhos de dados e controle;
- favorecer a reutilização de partes comuns às diferentes operações.

7. Regras de implementação

Não se deve utilizar `#delay` no RTL.

8. Verificação

A FPU deverá ser verificada utilizando Icarus Verilog e GTKWave.

A verificação deverá incluir:

- casos dirigidos, cobrindo as três operações;
- casos que produzam resultados exatos e resultados que exijam normalização ou arredondamento;
- pelo menos um caso de overflow, underflow e resultado zero;
- uma pequena campanha de vetores aleatórios, comparada com um modelo de referência.

O modelo de referência poderá utilizar `numpy.float32`.

9. Critérios de aprovação

- executar corretamente os casos dirigidos;
- apresentar resultados compatíveis com o modelo de referência nos vetores aleatórios, admitindo-se até 1 ULP de diferença;
- apresentar corretamente as flags de overflow e underflow;
- compilar e executar corretamente no Icarus Verilog;
- demonstrar o funcionamento da implementação por meio do testbench e/ou GTKWave.

10. Entrega e apresentação

Será realizada uma única entrega por equipe. A equipe deverá apresentar o projeto em aula.

11. Avaliação

Critério	Peso
Funcionamento da FPU – soma, subtração, multiplicação, normalização, arredondamento e tratamento de overflow/underflow	50%
Verificação – casos dirigidos, vetores aleatórios, testbench e análise dos resultados	30%
Arquitetura e implementação – modularidade e , organização do código	20%